Università di Pisa

Calcolo Numerico

Anno Accademico 2025–2026

Pietro Bacigalupi

17 marzo 2026

Indice

1 Teoria degli Errori **2**

1.1 Rappresentazione in Virgola Mobile 2

1.2 Teorema di Rappresentazione 2

1.3 Numeri di Macchina 3

1.4 Arrotondamento nelle macchine 4

1.5 Errore nel calcolo di funzione 5

1.5.1 Analisi dell'Errore nel Calcolo di Funzioni 6

1.6 Funzioni Razionali 8

1.6.1 Problemi ben/mal condizionati (come stimare ϵ_d) 9

1.6.2 Errori inerenti delle operazioni aritmetiche di base 11

1.7 Stimare l'errore algoritmico: 12

1.8 Problema Diretto 14

1.9 Funzioni Non Razionali 16

1 Teoria degli Errori

1.1 Rappresentazione in Virgola Mobile

Fissata una base $\beta > 1$, con $\beta \in \mathbb{N}$, un numero reale x può essere rappresentato nella forma:

$$x = \text{segno}(x) \cdot \beta^e \cdot \sum_{i=1}^{\infty} \alpha_i \, \beta^{-i} \tag{1}$$

dove:

- β è la base della rappresentazione;
- $e \in \mathbb{Z}$ è l'esponente;
- α_i sono le cifre della mantissa, tali che

$$0 \leq \alpha_i \leq \beta - 1 \tag{2}$$

e

$$\text{segno}(x) = \begin{cases} +1 & \text{se } x > 0 \\ -1 & \text{se } x < 0 \end{cases}$$

Attenzione

La rappresentazione così non è unica, ad esempio: $3{,}14 = 0{,}0314 \cdot 10^2$.

1.2 Teorema di Rappresentazione

Data $\beta > 1 \in \mathbb{N}$, $x \in \mathbb{R} \neq 0$ allora $\exists!$ rappresentazione di x in base β :

1. $\alpha_1 \neq 0$
2. $\nexists K \in \mathbb{N} : \alpha_j = \beta - 1 \forall j > k$

L'unica rappresentazione che verifica il teorema si dice **Rappresentazione in virgola mobile normalizzata**. Dove $\sum_{i=1}^{\infty} \alpha_i \, \beta^{-i}$ si dice **Mantissa** di $x \in \mathbb{R}$ e vale sempre che $\beta^{-1} < \sum_{i=1}^{\infty} \alpha_i \, \beta^{-i} < 1$

Esempio

La (1) esclude rappresentazioni con primo numero decimale uguale a 0:

- $0,\mathbf{0}123 \cdot 10^3$ Non va bene
- $1230,\mathbf{0} \cdot 10^0$ Non va bene
- $0,123 \cdot 10^5$ Va bene

La (2) esclude numeri in periodici con 9 finali (Esempio $1,4\overline{9}$, in base 10)

1.3 Numeri di Macchina

Problema : abbiamo inf cifre, ne possiamo tenere in considerazione solo un numero finito. Come facciamo? Tronchiamo la mantissa e limitiamo l'esponente.

Definizione : Dati, β, m, L, U, $\beta > 1$, $L \leq u$ e $m > 0$ definiamo **Insieme dei numeri di macchina** la funzione

$$F(\beta, m, L, U) = \left\{ x \in \mathbb{R} \;\middle|\; x = \text{segno}(x)\, \beta^e \sum_{i=1}^{m} \alpha_i \beta^{-i},\ L \leq e \leq U,\ \alpha_i \in \{0, 1, \ldots, \beta - 1\},\ \alpha_1 \neq 0 \right\} \cup \{0\}$$

I numeri di macchina sono un sottoinsieme dei numeri reali esattamente rappresentabili da un software

Esempio $\beta = 2$ $m = 3$ $L = -1$ $U = 3$ Ci sono un numero costante di elementi in $[\beta^{e-1}, \beta^e)$ e sono equispaziati.
Stiamo studiando quindi i **numeri macchina di base 2 con 3 cifre di mantissa** ed esponente da -1 a 3
Sono equispaziati in **scala logaritmica** negli intervalli

$$\left[2^i, 2^{i+1}\right)$$

quindi:

1. $[2^{-1}, 2^0) = [1/2, 1]$
2. $[2^0, 2^1) = [1, 2]$
3. $[2^1, 2^2) = [2, 4]$
4. $[2^2, 2^3) = [4, 8]$

Osservazione

Minimo Rappresentabile:

$$\beta^{\mathbf{L}} \cdot \alpha_{\mathbf{1}} \cdot \beta^{-\mathbf{1}} = \beta^{\mathbf{L-1}} \text{ dato che } \alpha_1 = 1$$

Massimo Rappresentabile:

$$\beta^U \cdot (\beta - 1) \sum_{j=1}^{m} \beta^{-j} = \beta^U \cdot (\beta - 1) \left(\frac{1 - \beta^{-m-1}}{1 - \beta^{-1}} - 1 \right) = \beta^U \left(1 - \beta^{-m} \right)$$

Quanti sono i numeri macchina rappresentabili:

$$\underbrace{2}_{\text{segno}} \quad \underbrace{(U - L + 1)}_{\text{esponente}} \quad \underbrace{\left(\beta^m - \beta^{m-1}\right)}_{\text{mantissa}} \quad \underbrace{+1}_{\{0\}}$$

1.4 Arrotondamento nelle macchine

Formato	β	m	L	U	Byte	$\varepsilon(precisione)$	$\lvert F \rvert$
Double	2	53	-1021	1024	8	2^{-52}	$2(2046)(2^{53} - 2^{52}) + 1$
Single	2	24	-124	128	4	2^{-23}	$2(253)(2^{24} - 2^{23}) + 1$

Tabella 1: Parametri IEEE 754 in virgola mobile

Osservazione Dato $x \in \mathbb{R}$, in generale $x \notin F(\beta, m, L, U)$. Si distinguono tre casi:

1. Se $|x| > \beta^U(1 - \beta^{-m})$ si ha **overflow**.
2. Se $0 < |x| < \beta^{L-1}$ si ha **underflow**.
3. Se
$$\beta^{L-1} \leq |x| \leq \beta^U(1 - \beta^{-m}),$$
il numero è rappresentabile in ordine di grandezza ma può richiedere **arrotondamento**.

Definizione Si definisce **funzione di arrotondamento** una funzione

$$\mathrm{RD} : \mathbb{R} \longrightarrow F(\beta, m, L, U)$$

che ad ogni numero reale x associa un numero macchina $\mathrm{RD}(x)$ appartenente all'insieme $F(\beta, m, L, U)$.

Le scelte più comuni sono:

1. **Troncamento**: $TR(x) = \lfloor x \rfloor \forall x$
2. **Round-UP**: $UP(x) = \lceil x \rceil \forall x$
3. **Round-To-Nearest**:

$$\text{RN(x)} = \begin{cases} \lfloor x \rfloor \text{ se } \alpha_{m+1} < \frac{\beta}{2} \\ \lceil x \rceil \text{ se } \alpha_{m+1} \geq \frac{\beta}{2} \end{cases}$$

Quella usata dai calcolatori, che useremo noi, è la **(3)**

Definizione Sia $x \in \mathbb{R}$ e sia $\text{RN}(x)$ il numero macchina ottenuto mediante arrotondamento al più vicino (round-to-nearest).

Si definisce **errore assoluto di arrotondamento** la quantità

$$\delta(x) = x - \text{RN}(x).$$

Si definisce **errore relativo di arrotondamento** la quantità

$$\varepsilon(x) = \frac{\delta(x)}{x}, \qquad x \neq 0.$$

Nel caso di arrotondamento al più vicino vale la stima

$$|\delta(x)| \leq \frac{1}{2}\,\beta^{e-m}.$$

Poiché $|x| \geq \beta^{e-1}$, si ottiene

$$|\varepsilon(x)| = \frac{|\delta(x)|}{|x|} \leq \frac{\frac{1}{2}\beta^{e-m}}{\beta^{e-1}} = \frac{1}{2}\beta^{1-m}.$$

Si osserva che il termine finale non dipende da x.

La quantità

$$u = \frac{1}{2}\beta^{1-m}$$

è detta **precisione di macchina** (o *unit roundoff*). In doppia precisione (IEEE 754):

$$u = 2^{-53} \approx 1.11 \times 10^{-16}.$$

1.5 Errore nel calcolo di funzione

Le operazioni di macchina sono 4 e sono $\otimes \oplus \ominus \oslash$ dove

- $x \oplus y = RN(x + y)$

- $x \ominus y = RN(x - y)$
- $x \otimes y = RN(x \cdot y)$
- $x \oslash y = RN(x \div y)$

Non è valida la proprietà associativa e distributiva

1.5.1 Analisi dell'Errore nel Calcolo di Funzioni

Sia $f : \mathbb{R}^m \to \mathbb{R}$ una funzione matematica che vogliamo valutare. Si vuole calcolare il valore $f(p_0)$, dove il punto di partenza è un vettore di m componenti:

$$p_0 = (x_1^{(0)}, x_2^{(0)}, \dots, x_m^{(0)}) \in \mathbb{R}^m$$

Fonti dell'errore: Nel passaggio dalla teoria alla pratica, incontriamo tre ostacoli principali:

1. **Approssimazione della funzione**: La funzione teorica f viene sostituita da una funzione $\tilde{f}$ che il calcolatore sa gestire (ovvero che usa solo operazioni elementari). *Esempio: calcolare e^x usando solo i primi termini dello sviluppo di Taylor.*
2. **Algoritmizzazione**: La funzione $\tilde{f}$ viene tradotta in un algoritmo $\tilde{f}_a$. Qui entrano in gioco le operazioni di macchina $\oplus, \ominus, \otimes, \oslash$, ognuna delle quali introduce un errore di arrotondamento.
3. **Arrotondamento dei dati**: Il punto p_0 originale spesso non è rappresentabile esattamente in memoria (pensa a π o a numeri con troppe cifre). Viene quindi sostituito dal numero di macchina più vicino: $p_1 = \mathrm{RN}(p_0)$.

Conclusione: Il nostro obiettivo sarebbe ottenere $f(p_0)$, ma ciò che il computer ci restituisce effettivamente è $\tilde{f}_a(p_1)$.

Esempio: Calcolo di e^π Vogliamo calcolare il valore della funzione $f(x) = e^x$ nel punto $p_0 = \pi$.

1. **Approssimazione matematica**: Poiché il calcolatore non può calcolare la funzione esponenziale esatta, usiamo uno sviluppo di Taylor troncato al secondo ordine:

$$f(x) \approx 1 + x + \frac{x^2}{2} =: \tilde{f}(x)$$

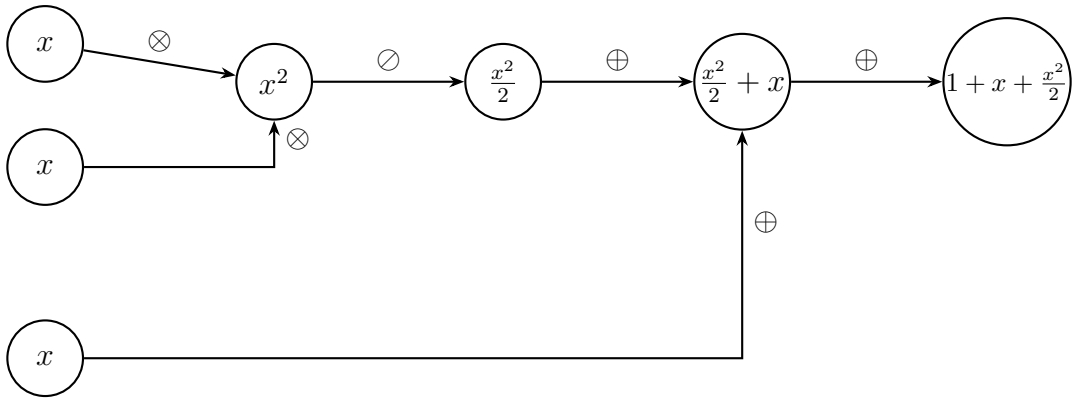
2. **Arrotondamento dell'input**: Il numero π ha infinite cifre. Lo arrotondiamo a un numero di macchina p_1 (nell'esempio con 5 cifre):

$$p_1 = RN(\pi) = 3.1415$$

3. **Esecuzione dell'algoritmo**: Implementiamo la formula usando solo operazioni di macchina. L'ordine scelto definisce l'algoritmo $\tilde{f}_a(p_1)$:

$$\tilde{f}_a(p_1) = 1 \oplus (p_1 \oplus ((p_1 \otimes p_1) \oslash 2))$$

Grafo computazionale: L'algoritmo sopra descritto può essere visualizzato tramite il seguente schema di flusso:



Analisi e Spiegazione del Grafo Computazionale Il grafo sopra riportato rappresenta la struttura logica e sequenziale dell'algoritmo $\tilde{f}_a(p_1)$. In analisi numerica, questo schema è fondamentale per visualizzare come l'errore si genera e si propaga attraverso i calcoli.

- **Nodi di Input (Sorgenti)**: I cerchi a sinistra contenenti x rappresentano il numero di macchina $p_1 = RN(\pi)$. Nel grafo, lo stesso valore x viene "pescato" tre volte dalla memoria per alimentare i diversi rami del calcolo.
- **Nodi Operatori (Cerchi con $\otimes, \oslash, \oplus$)**: Ogni cerchio che contiene un simbolo operativo rappresenta un'operazione di macchina.
  - Il primo nodo esegue $x \otimes x$, producendo il risultato parziale x^2.
  - Il secondo nodo riceve x^2 e lo divide per la costante 2 tramite l'operatore $\oslash$.
  - Il terzo e il quarto nodo eseguono le somme ($\oplus$) necessarie per completare il polinomio di Taylor.

- **Propagazione dell'Errore locale**: Ogni freccia che entra in un operatore porta con sé un valore potenzialmente già viziato da errori precedenti. Ogni nodo operatore, a sua volta, introduce un nuovo **errore di arrotondamento locale** dovuto alla funzione $RN(\cdot)$, poiché il risultato di un'operazione tra numeri di macchina non è necessariamente un numero di macchina.

- **Significato Algoritmico**: Il grafo evidenzia che l'algoritmo non calcola la funzione "tutta insieme", ma come una catena di eventi. L'errore totale δ_f è dunque la somma accumulata degli errori nati in ogni singolo nodo del grafo, influenzata pesantemente dall'ordine in cui le operazioni sono collegate tra loro.

Definizione: Errore Totale

Data $f : \mathbb{R}^m \to \mathbb{R}$, un punto p_0 e un algoritmo $\tilde{f}_a$, l'errore totale è dato da:

$$\delta_f = \tilde{f}_a(p_1) - f(p_0) \quad \text{dove } p_1 = RN(p_0)$$

1.6 Funzioni Razionali

Per semplificare l'analisi, assumiamo che f sia una **funzione razionale**, ovvero definibile mediante un numero finito di operazioni elementari $\{+, -, \times, /\}$. In questo caso, la funzione approssimata coincide con quella teorica: $\tilde{f}(x) = f(x)$.

Errore Assoluto L'errore assoluto totale δ_f può essere scomposto come segue:

$$\begin{aligned}
\delta_f &= f_a(p_1) - f(p_0) \\
&= f_a(p_1) - f(p_1) + f(p_1) - f(p_0) \\
&= \delta_a + \delta_d
\end{aligned}$$

dove:

- $\delta_a = f_a(p_1) - f(p_1)$ è l'**errore algoritmico**;
- $\delta_d = f(p_1) - f(p_0)$ è l'**errore inerente**.

Errore Relativo L'errore relativo ϵ_f si ottiene dividendo l'errore assoluto per il valore esatto:

$$\begin{aligned}
\epsilon_f &= \frac{\delta_f}{f(p_0)} = \frac{f_a(p_1) - f(p_0)}{f(p_0)} \\
&= \frac{\delta_a}{f(p_0)} + \frac{\delta_d}{f(p_0)} \\
&= \frac{f_a(p_1) - f(p_1)}{f(p_1)} \cdot \frac{f(p_1)}{f(p_0)} + \epsilon_d \\
&= \epsilon_a \left(1 + \frac{f(p_1) - f(p_0)}{f(p_0)}\right) + \epsilon_d \\
&= \epsilon_a(1 + \epsilon_d) + \epsilon_d
\end{aligned}$$

Sviluppando il prodotto si ottiene $\epsilon_a + \epsilon_a\epsilon_d + \epsilon_d$. Poiché ϵ_a ed ϵ_d sono quantità molto piccole, il loro prodotto $\epsilon_a\epsilon_d$ è trascurabile (ordine superiore). Pertanto:

$$\epsilon_f \approx \epsilon_a + \epsilon_d \tag{3}$$

In generale, per limitare $|\delta_f|$ si cercano disuguaglianze del tipo:

$$|\delta_a| < \tau_1, \quad |\delta_d| < \tau_2 \implies |\delta_f| < \tau_1 + \tau_2$$

1.6.1 Problemi ben/mal condizionati (come stimare ϵ_d)

Errore assoluto inerente: Definiamo l'errore assoluto inerente come:

$$\delta_d := f(p_1) - f(p_0)$$

Sotto l'ipotesi che $f \in C^1(D), D \subseteq \mathbb{R}^m$ e $p_0 \in D$, si può sfruttare lo **sviluppo di Taylor** di f centrato in p_0 troncato al primo ordine:

$$\begin{aligned}
f(p_1) - f(p_0) &= f(p_0) + \nabla f(\widetilde{p})^T(p_1 - p_0) - f(p_0) \quad \text{dove } \widetilde{p} \in \overline{p_1 p_0} \\
&= \nabla f(\widetilde{p})^T(p_1 - p_0) = \sum_{j=1}^{m} \frac{\partial f}{\partial x_j}(\widetilde{p}) \cdot (x_j^{(1)} - x_j^{(0)}) \\
&\approx \sum_{j=1}^{m} \frac{\partial f}{\partial x_j}(p_0) \cdot (x_j^{(1)} - x_j^{(0)})
\end{aligned}$$

Definendo $\delta_j = (x_j^{(1)} - x_j^{(0)})$ come l'errore sulla componente j-esima di p_0, otteniamo la stima:

$$\delta_d \approx \sum_{j=1}^{m} \frac{\partial f}{\partial x_j}(p_0) \cdot \delta_j$$

Coefficienti di amplificazione: Le derivate parziali $\frac{\partial f}{\partial x_j}(p_0)$ agiscono come **coefficienti di amplificazione** dell'errore.

- Se una derivata è molto grande in modulo, anche un piccolo errore di arrotondamento δ_j nella componente j dell'input può portare a un errore inerente δ_d molto grande nel risultato finale.
- In questo caso, diciamo che il problema è **mal condizionato**.

Errore relativo inerente: L'errore relativo inerente ϵ_d è definito come il rapporto tra l'errore assoluto inerente e il valore esatto della funzione nel punto p_0:

$$\epsilon_d := \frac{f(p_1) - f(p_0)}{f(p_0)} \tag{4}$$

Sostituendo la stima del primo ordine ottenuta con Taylor, possiamo riscrivere l'espressione per evidenziare gli errori relativi sulle singole componenti ϵ_j:

$$\begin{aligned}\epsilon_d \approx \frac{\sum_{j=1}^{m} \frac{\partial f}{\partial x_j}(p_0) \cdot \delta_j}{f(p_0)} &= \sum_{j=1}^{m} \frac{x_j^{(0)}}{f(p_0)} \frac{\partial f}{\partial x_j}(p_0) \cdot \frac{x_j^{(1)} - x_j^{(0)}}{x_j^{(0)}} \\ &= \sum_{j=1}^{m} \underbrace{\left(\frac{x_j^{(0)}}{f(p_0)} \frac{\partial f}{\partial x_j}(p_0) \right)}_{\rho_j} \cdot \epsilon_j\end{aligned}$$

dove $\epsilon_j = \frac{x_j^{(1)} - x_j^{(0)}}{x_j^{(0)}}$ rappresenta l'errore relativo sulla componente j-esima di p_0.

$$\epsilon_d \approx \sum_{j=1}^{m} \epsilon_j \rho_j \quad \text{dove } \rho_j := \frac{x_j^{(0)}}{f(p_0)} \frac{\partial f}{\partial x_j}(p_0)$$

I termini ρ_j sono detti **coefficienti di amplificazione** (o coefficienti di condizionamento relativo).

Condizionamento del problema: In base al modulo dei coefficienti ρ_j, possiamo classificare la stabilità del calcolo della funzione rispetto alle perturbazioni dei dati di input:

- **Problema mal condizionato**: Si verifica quando i coefficienti ρ_j sono grandi in modulo (o ne esiste almeno uno molto grande). In questo caso, anche piccoli errori relativi nei dati ϵ_j vengono fortemente amplificati nel risultato finale.

- **Problema ben condizionato**: Si verifica quando i coefficienti ρ_j sono "piccoli" (tipicamente vicini a 1). In tale situazione, l'errore relativo finale ϵ_d risulta dello stesso ordine di grandezza degli errori sui dati ϵ_i.

1.6.2 Errori inerenti delle operazioni aritmetiche di base

Consideriamo il caso in cui la funzione f rappresenti un'operazione aritmetica elementare tra due variabili ($m = 2$), ovvero $f(x, y) = x$ op y. Nella tabella seguente sono riportati l'errore assoluto inerente δ_d e l'errore relativo inerente ϵ_d per le quattro operazioni di macchina.

Operatore	**Errore inerente** δ_d	**Errore inerente relativo** ϵ_d
$x \oplus y$	$\delta_x + \delta_y$	$\frac{x}{x+y}\epsilon_x + \frac{y}{x+y}\epsilon_y$
$x \ominus y$	$\delta_x - \delta_y$	$\frac{x}{x-y}\epsilon_x - \frac{y}{x-y}\epsilon_y$
$x \otimes y$	$y\delta_x + x\delta_y$	$\epsilon_x + \epsilon_y$
$x \oslash y$	$\frac{1}{y}\delta_x - \frac{x}{y^2}\delta_y$	$\epsilon_x - \epsilon_y$

Tabella 2: Analisi dell'errore inerente per le operazioni elementari ($m = 2$)

Osservazioni sul condizionamento: Dalla tabella si possono trarre conclusioni fondamentali sulla stabilità delle operazioni:

- **Moltiplicazione e Divisione**: Sono operazioni sempre **ben condizionate**, poiché i coefficienti di amplificazione sono unitari ($\rho_x = 1, \rho_y = \pm 1$). L'errore relativo finale è semplicemente la somma o differenza degli errori relativi iniziali.

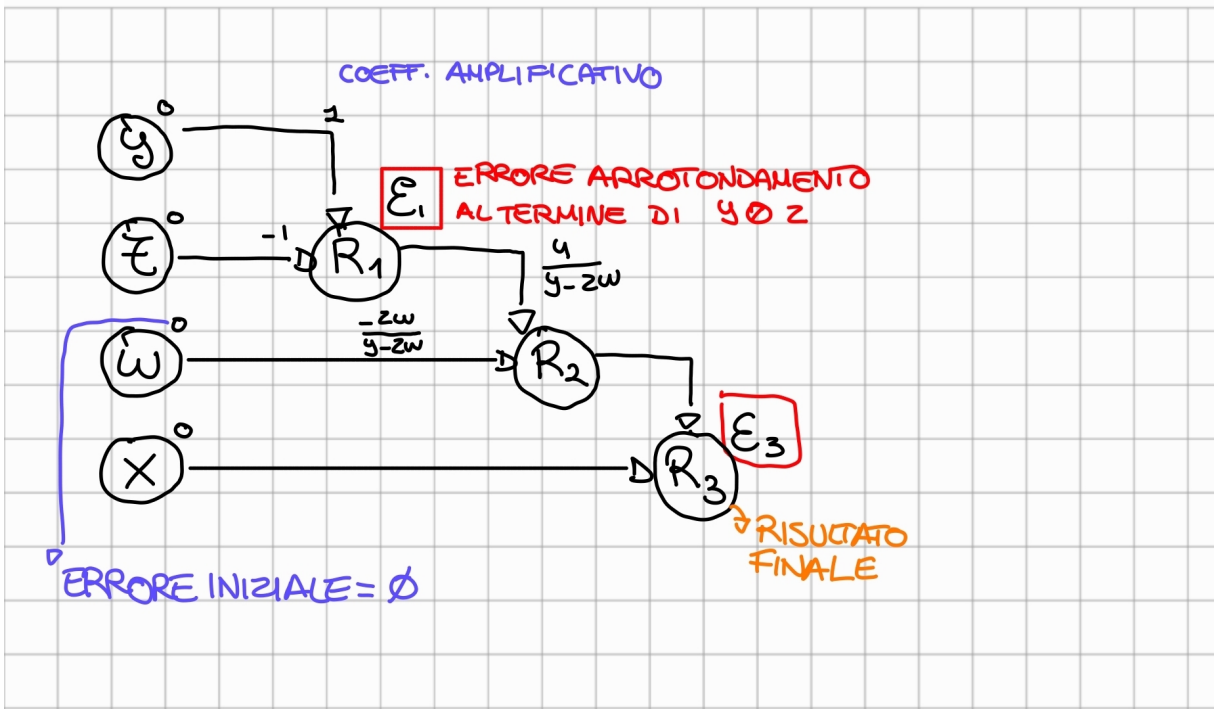
- **Addizione e Sottrazione**: Il condizionamento dipende dai valori di x e y. In particolare, nella sottrazione $x \ominus y$, se $x \approx y$, il denominatore $(x - y)$ tende a zero, rendendo i coefficienti ρ_j estremamente grandi. Questo fenomeno è noto come **cancellazione numerica** ed è una delle principali cause di mal condizionamento nei calcoli.

1.7 Stimare l'errore algoritmico:

Fissato un algoritmo $f_a(x)$ possiamo stimare $\delta_a = f(p_1) - f_a(p_1), \epsilon_a = \frac{f(p_1)-f_a(p_1)}{f(p_1)}$ (Errore assoluto e relativo algoritmico). Assumiamo che non ci sia errore sui dati iniziali dato che $p_1 \in F(\beta, m, l, U)$ e studiamo come le operazioni di macchina propagano l'errore di arrotondamento sui risultati intermedi.

Esempi pratici:

$$f(x, y, z, w) = x(\frac{y}{z} - w) \quad f_a(x) = x \otimes ((y \oslash z) \ominus w)$$



Spiegazione

- Sulla sinistra ci sono i valori iniziali y^0, z^0, w^0, x^0. In questa analisi ci si concentra solo sull'errore introdotto dall'algoritmo, quindi assumiamo che i dati in ingresso siano esatti.
- Il calcolatore esegue la prima operazione: $y \oslash z$.
  - **Risultato intermedio** ($r1$): Rappresenta il valore della divisione memorizzato in memoria
  - **Errore locale** (ε_1): Poichè il risultato di $y \div z$ potrebbe avere infinite cifre, il computer deve arrotondarlo. Questo arrotondamento genera l'errore locale ε_1

- **Coefficienti sui rami**: Sulle frecce che entrano in r_1 i valori 1 e -1 indicano l'errore che si propaga nella divisione.

- Il valore ottenuto (r_1) viene sottratto a w : $(r_1 \ominus w)$.
  - **Risultato intermedio (r_2)**: Adesso è il risultato della parentesi tonda
  - **Errore locale (ε_2)** Operazione di sottrazione introduce un nuovo errore di arrotondamento proprio, indicato con ε_2
  - **Coefficienti di amplificazione**: Se il denominatore $(y - zw)$ è molto piccolo (ovvero se $\frac{y}{z} \approx w$), quesi coefficienti diventano enormi, causando il fenomeno della **cancellazione numerica** (problema mal condizionato).

- Infine, il risultato (r_2) viene moltiplicato per x : $x \otimes r_2$.
  - **Risultato finale (r_3)**: Valore definitivo restituito dal calcolatore.
  - **Errore locale (ε_3)** Anche quest'ultima moltiplicazione introduce un piccolo errore di arrotondamento finale.
  - **Propagazione**: La moltiplicazione per sè non amplifica ulteriormente gli errori precedenti in modo drammatico.

$$
\begin{aligned}
\epsilon_{r_3} &= \epsilon_3 + 1\epsilon_x + 1\epsilon_{r_2} = \epsilon_3 + \epsilon_{r_2} \\
&= \epsilon_3 + \epsilon_2 + \left(\frac{-zw}{y - zw}\right)\epsilon_w + \frac{y}{y - zw}\epsilon_{r_1} = \\
&= \epsilon_3 + \epsilon_2 + \frac{y}{y - zw}\epsilon_{r_1} = \epsilon_3 + \epsilon_2 + \frac{y}{y - zw}(\epsilon_1 + 1\epsilon_y - 1\epsilon_z) = \\
&= \epsilon_3 + \epsilon_2 + \frac{y}{y - zw}\epsilon_1 = \epsilon_a \quad \text{Sapendo che } \epsilon_w, \epsilon_y, \epsilon_x, \epsilon_z = 0
\end{aligned}
$$

Vale che $|\epsilon_1|, |\epsilon_2|, |\epsilon_3| \leq u$.
Nel caso di errori assoluti: $|\delta_1|, |\delta_2|, |\delta_3| \leq u \cdot \max\{|x|, |y|, |z|\}$

Esempio $f(x, y) = x^2 - y^2$

Algoritmo 1:

$r_1 = x \cdot x$

$r_2 = y \cdot y$

$r_3 = r_1 - r_2$

$$\epsilon_a = \epsilon_3 + \frac{x^2}{x^2 - y^2}\epsilon_1 - \frac{y^2}{x^2 - y^2}\epsilon_2$$

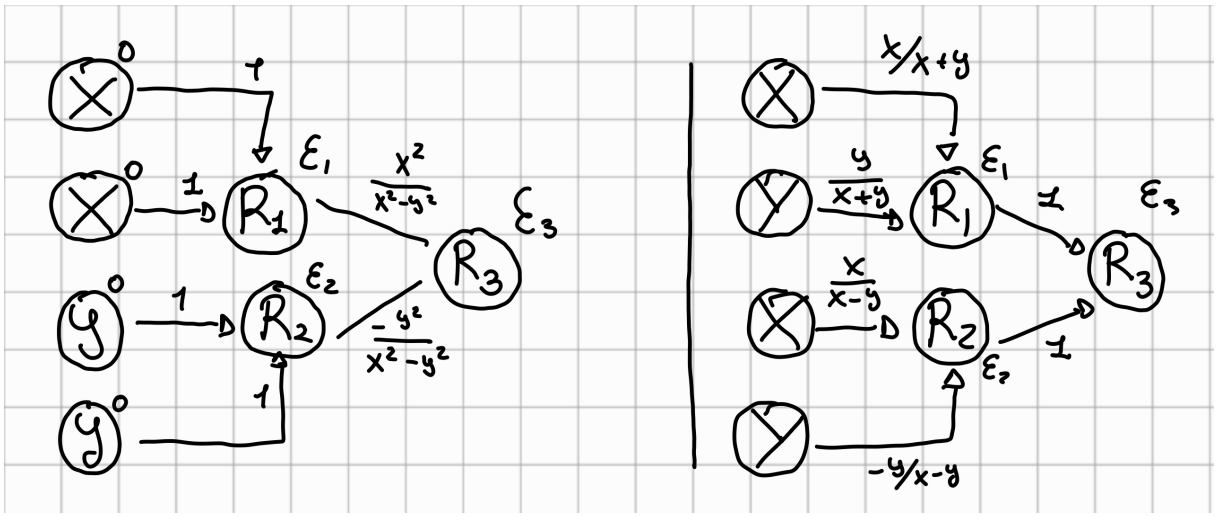
Algoritmo 2:

$r_1 = x + y$

$r_2 = x - y$

$r_3 = r_1 \cdot r_2$

$$\epsilon_a = \epsilon_1 + \epsilon_2 + \epsilon_3$$



Nota tecnica:
L'**Algoritmo 1** è numericamente instabile quando $x \approx y$ a causa del fenomeno della *cancellazione numerica* (i coefficienti dell'errore ϵ_1 ed ϵ_2 tendono all'infinito).
L'**Algoritmo 2** è invece numericamente stabile poiché l'errore generato ϵ_a è semplicemente la somma degli errori locali di arrotondamento, indipendentemente dai valori di x e y.

1.8 Problema Diretto

Le tecniche viste per stimare ϵ_a e ϵ_d le possiamo usare per stimare

$$|\epsilon_f| \leq |\epsilon_d| + |\epsilon_a|$$
$$|\delta_f| \leq |\delta_d| + |\delta_a|$$

Problema diretto: Date f, una stima degli errori δ_{x_i} e un algoritmo, vogliamo stimare δ_f per $p_0 \in D \subseteq \mathbb{R}^m$

Esempio

Consideriamo la funzione $f(x_1, x_2) = \frac{x_1}{x_2}$ definita in un dominio D tale che $x_1 \in [1, 3]$ e $x_2 \in [4, 5]$ ($D = [1, 3] \times [4, 5], p_0 \in D$). Supponiamo che l'errore sui dati sia limitato da $|\delta_{x_i}| < \frac{1}{2} \cdot 10^{-2}$ per $i = 1, 2$.

Errore inerente: Vogliamo determinare una maggiorazione per l'errore inerente δ_d. Calcoliamo le derivate parziali e i loro estremi superiori nel dominio D:

- Rispetto a x_1:

$$\frac{\partial f}{\partial x_1} = \frac{1}{x_2} \implies A_1 = \sup_{x \in D} \left| \frac{\partial f}{\partial x_1}(x) \right| = \sup_{(x_1,x_2) \in D} \left| \frac{1}{x_2} \right| = \frac{1}{4}$$

- Rispetto a x_2:

$$\frac{\partial f}{\partial x_2} = -\frac{x_1}{x_2^2} \implies A_2 = \sup_{x \in D} \left| \frac{\partial f}{\partial x_2}(x) \right| = \sup_{(x_1,x_2) \in D} \left| -\frac{x_1}{x_2^2} \right| = \frac{3}{16}$$

Dunque, la stima dell'errore totale basata sulla scomposizione $|\delta_f| \leqslant |\delta_a| + |\delta_d|$ diventa:

$$|\delta_f| \leqslant |\delta_a| + A_1|\delta_{x_1}| + A_2|\delta_{x_2}| = |\delta_a| + \frac{1}{4} \cdot \frac{1}{2} \cdot 10^{-2} + \frac{3}{16} \cdot \frac{1}{2} \cdot 10^{-2}$$

Errore algoritmico: L'errore algoritmico può essere stimato usando la linearizzazione:

$$\delta_a = \sum_{j=1}^{m} \frac{\partial f}{\partial x_j}(p_0) \cdot \delta_j \implies |\delta_a| \leqslant \sum_{j=1}^{m} \sup_{p \in D} \left| \frac{\partial f}{\partial x_j}(p) \right| \cdot |\delta_j|$$

In questo caso specifico (divisione di macchina), l'errore algoritmico è dato dall'arrotondamento del risultato:

$$|\delta_a| = \left| \frac{x_1}{x_2} \right| \cdot \frac{1}{2} \cdot 10^{-2} \quad \text{(dato solo dall'errore di troncamento)}$$

Dunque l'errore assoluto: Combinando i risultati ottenuti:

$$|\delta_f| \leqslant \left| \frac{x_1}{x_2} \right| \cdot \frac{1}{2} \cdot 10^{-2} + \frac{1}{4} \cdot \frac{1}{2} \cdot 10^{-2} + \frac{3}{16} \cdot \frac{1}{2} \cdot 10^{-2} = \frac{23}{32} \cdot 10^{-2} \leqslant 1$$

Problema Inverso

Problema inverso: dato $\tau > 0$, f e $p_0 \in \mathbb{R}^m$, determinare un algoritmo e un valore di precisione di macchina u tale per cui:

$$|\delta_f| < \tau$$

Esempio Sia $f(x_1, x_2) = \frac{x_1}{x_2}$ con dominio $D = [1, 2] \times [-2, -1]$. Si vuole trovare un algoritmo e una precisione u per cui $|\delta_f| < 10^{-2}$.

Scegliamo $x_1 \oslash x_2$ come algoritmo e stimiamo gli errori sui dati $|\delta_{x_1}|, |\delta_{x_2}|$:

$$|\delta_{x_i}| \leqslant u \cdot |x_i| \leqslant 2u, \quad i = 1, 2$$

Calcoliamo i coefficienti di amplificazione (derivate parziali):

- $A_1 = \sup_D \left|\frac{\partial f}{\partial x_1}\right| = \sup_D \left|\frac{1}{x_2}\right| = 1$
- $A_2 = \sup_D \left|\frac{\partial f}{\partial x_2}\right| = \sup_D \left|-\frac{x_1}{x_2^2}\right| = 2$

Stima dell'errore totale: Utilizzando la scomposizione dell'errore e le maggiorazioni trovate:

$$\begin{aligned}|\delta_f| &\leqslant |\delta_a| + \underbrace{2u}_{|\delta_{x_1}|\cdot A_1} + \underbrace{2u}_{|\delta_{x_2}|\cdot A_2} \\ &\leqslant \max_D \left|\frac{x_1}{x_2}\right| \cdot u + 4u = 2u + 4u = 6u\end{aligned}$$

Determinazione di u: Per soddisfare la richiesta del problema:

$$|\delta_f| \leqslant 6u \leqslant 10^{-2} \implies u = \frac{1}{6} \cdot 10^{-2}$$

1.9 Funzioni Non Razionali

Se si deve valutare un'espressione matematica **non razionale** (ad esempio $f(x, y) = e^{\cos(x+y)}$), il calcolatore non può calcolarla esattamente. Si procede quindi scegliendo un'approssimazione razionale $\tilde{f}(x) \approx f(x)$ (ad esempio tramite uno sviluppo di Taylor) e si valuta $\tilde{f}(p_0)$.

A causa dell'uso dell'aritmetica floating point e dell'approssimazione della funzione, quello che si ottiene effettivamente è $\tilde{f}_a(p_1)$. Si osserva che l'errore totale può essere scomposto come:

$$f(p_0) - \tilde{f}_a(p_1) = \underbrace{f(p_0) - f(p_1)}_{\delta_d} + \underbrace{f(p_1) - \tilde{f}(p_1)}_{\delta_{an}} + \underbrace{\tilde{f}(p_1) - \tilde{f}_a(p_1)}_{\delta_a}$$

Dove i contributi sono:

- δ_d: **Errore inerente** sulla funzione originale f;
- δ_{an}: **Errore analitico** (dovuto all'approssimazione di f con $\tilde{f}$);
- δ_a: **Errore algoritmico** sulla funzione approssimata $\tilde{f}$.

$$\boxed{\delta_f = \delta_d + \delta_a + \delta_{an}}$$

Osservazione: L'errore inerente è calcolato sulla funzione originale (f), mentre l'errore algoritmico è calcolato sulla funzione approssimata $\tilde{f}$ (di conseguenza, per stimare δ_a serve un'analisi del grafo computazionale dell'algoritmo che valuta effettivamente $\tilde{f}$).

Con tecniche simili a quanto visto nel caso razionale, si dimostra che (a meno di termini trascurabili di ordine superiore) per l'**errore relativo** vale:

$$\epsilon_f := \frac{f(p_0) - \tilde{f}_a(p_1)}{f(p_0)} \qquad \epsilon_f \approx \epsilon_d + \epsilon_a + \epsilon_{an} \tag{5}$$

dove l'errore relativo analitico è definito come:

$$\epsilon_{an} := \frac{f(p_1) - \tilde{f}(p_1)}{f(p_1)} \tag{6}$$

Stima dell'errore analitico Le quantità δ_{an} ed ϵ_{an} si stimano utilizzando i risultati della **teoria dell'approssimazione**. L'esempio più comune (e l'unico che verrà trattato nel corso) è l'approssimazione di una funzione mediante il **polinomio di Taylor**, per funzioni del tipo $f : \mathbb{R} \to \mathbb{R}$.

Reminder: Data una funzione $f : \mathbb{R} \to \mathbb{R}$, con $f \in C^{n+1}(\mathbb{R})$ e un punto di centro $x_0 \in \mathbb{R}$, allora:

$$f(x) = \underbrace{\sum_{j=0}^{n} \frac{f^{(j)}(x_0)}{j!} \cdot (x - x_0)^j}_{\text{Polinomio di grado } n} + \underbrace{\frac{f^{(n+1)}(\eta)}{(n+1)!}(x - x_0)^{n+1}}_{\text{Resto di Lagrange}} \tag{7}$$

Dove:

- La parte polinomiale sarà la nostra **funzione razionale** $\tilde{f}(x)$ che usiamo per approssimare $f(x)$.
- Il **Resto di Lagrange** è il termine che serve per stimare l'errore analitico.
- η è un valore incognito compreso nell'intervallo tra x e x_0 (ovvero $\eta \in [x, x_0]$ oppure $\eta \in [x_0, x]$).

Osservazione: In questo contesto, l'errore analitico assoluto e relativo si identificano direttamente con il resto della serie di Taylor:

$$\delta_{an}(x) = \frac{f^{(n+1)}(\eta)}{(n+1)!}(x - x_0)^{n+1} \tag{8}$$

$$\epsilon_{an}(x) = \frac{f^{(n+1)}(\eta)}{(n+1)!} \frac{(x - x_0)^{n+1}}{f(x)} \tag{9}$$

Esempio: Approssimazione della Funzione Esponenziale Consideriamo l'approssimazione della funzione $f(x) = e^x$ mediante il suo polinomio di Taylor di grado $n = 2$

centrato in $x_0 = 0$:

$$\tilde{f}(x) = 1 + x + \frac{x^2}{2}$$

Vogliamo stimare quanto sia grande l'errore relativo analitico $|\epsilon_{an}|$ e trovare una sua maggiorazione per $x \in [-1, 1]$.

Calcolo dell'errore analitico assoluto: Utilizzando il resto di Lagrange con $n = 2$ (dove la derivata terza di e^x è ancora e^x):

$$\delta_{an}(x) = e^x - \left(1 + x + \frac{x^2}{2}\right) = \frac{f^{(3)}(\eta)}{3!}x^3 = \frac{e^\eta x^3}{6}$$

con η compreso tra 0 e x.

Calcolo dell'errore analitico relativo: Dividendo per il valore esatto della funzione $f(x) = e^x$:

$$\epsilon_{an}(x) = \frac{\delta_{an}(x)}{f(x)} = \frac{e^\eta x^3}{6e^x}$$

Maggiorazione dell'errore nell'intervallo $[-1, 1]$: Per trovare un limite superiore, cerchiamo i valori massimi dei componenti per $x, \eta \in [-1, 1]$:

$$\max_{x\in[-1,1]} |\epsilon_{an}(x)| \leqslant \max_{\eta\in[-1,1]} e^\eta \cdot \max_{x\in[-1,1]} \left|\frac{x^3}{6e^x}\right| \leqslant \frac{e}{6} \cdot \frac{\max_{x\in[-1,1]} |x^3|}{\min_{x\in[-1,1]} e^x}$$

Sostituendo i valori estremi nel dominio:

$$|\epsilon_{an}(x)| \leqslant \frac{e}{6} \cdot \frac{1}{e^{-1}} = \frac{e^2}{6} \approx 1.23$$

Conclusione: L'errore ottenuto ($\approx 123\%$) non è molto accurato. Questo indica che un polinomio di secondo grado non è sufficiente per approssimare bene l'esponenziale su un intervallo ampio come $[-1, 1]$.

Generalizzazione al Grado n Se considerassimo il polinomio di Taylor di grado n, otterremmo un errore assoluto analitico espresso come:

$$\delta_{an}(x) = \frac{e^\eta x^{n+1}}{(n+1)!} \tag{10}$$

Ragionando come nel caso precedente per la maggiorazione dell'errore relativo nell'intervallo $x \in [-1, 1]$, si ottiene:

$$|\epsilon_{an}(x)| \leqslant \frac{e^2}{(n+1)!} \xrightarrow[n \to +\infty]{} 0 \tag{11}$$

Questo limite ci conferma che all'aumentare del grado del polinomio, l'approssimazione diventa sempre più precisa, tendendo a zero.

Esempio con $n = 4$ Considerando il polinomio di quarto grado:

$$\tilde{f}(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \frac{x^4}{24}$$

Per $x \in [-1, 1]$, si ottiene una stima dell'errore molto più contenuta:

$$|\epsilon_{an}(x)| \leqslant \frac{e^2}{5!} = \frac{e^2}{120} \approx 0,061 \tag{12}$$

Conclusione finale: Passando dal grado $n = 2$ al grado $n = 4$, l'errore relativo è sceso da circa $1,23$ (123%) a $0,061$ ($6,1\%$). Questo rende l'approssimazione decisamente più accettabile per fini computazionali.